



Evaluation of Genetic Algorithms in Multi-Vehicle Control for Multi-Target Tracking Applications

Aidan Blair
aidan.blair@rmit.edu.au
RMIT University
Melbourne, VIC, Australia

Amirali Khodadadian Gostar
amirali.khodadadian@rmit.edu.au
RMIT University
Melbourne, VIC, Australia

Xiaodong Li
xiaodong.li@rmit.edu.au
RMIT University
Melbourne, VIC, Australia

Alireza Bab-Hadiashar
alireza.bab-hadiashar@rmit.edu.au
RMIT University
Melbourne, VIC, Australia

Reza Hoseinnezhad
reza.hoseinnezhad@rmit.edu.au
RMIT University
Melbourne, VIC, Australia

ABSTRACT

Tracking multiple dynamic targets using a network of sensors is a challenging yet essential task in intelligent vehicles, that requires positioning the sensors in optimal locations to collect informative measurements of the system. This paper studies the application of genetic algorithms (GAs) in solving this problem. Multiple GA-based multi-sensor control algorithms are proposed, tested and compared against existing baseline methods on challenging simulated scenarios. We find that the proposed solutions significantly enhance tracking performance and computational tractability compared to the baseline methods, showcasing the suitability of GA-based control methods in intelligent transportation systems.

CCS CONCEPTS

• **Computing methodologies** → **Motion path planning**; **Distributed algorithms**; • **Applied computing** → **Physical sciences and engineering**.

KEYWORDS

genetic algorithm, multi-target tracking, vehicle control, distributed sensor network, random finite sets

ACM Reference Format:

Aidan Blair, Amirali Khodadadian Gostar, Xiaodong Li, Alireza Bab-Hadiashar, and Reza Hoseinnezhad. 2024. Evaluation of Genetic Algorithms in Multi-Vehicle Control for Multi-Target Tracking Applications. In *Genetic and Evolutionary Computation Conference (GECCO '24 Companion)*, July 14–18, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3638530.3654261>

1 INTRODUCTION

Tracking multiple dynamic targets with large networks of sensor-equipped intelligent vehicles has gained significant attention in various fields, including autonomous vehicles [12], space debris monitoring, and defense. In the Bayesian filtering framework, each

sensor observes the targets within its field-of-view (FoV) and executes a stochastic multi-object filter on board, producing a multi-object posterior that is broadcast through the network and fused locally at each sensor node. Multi-sensor control aims to coordinate multiple sensors to obtain informative and reliable observations that are crucial in accurately tracking a large number of targets.

Distributed networks are the preferred option for situational awareness applications, such as multi-sensor control, due to the reduced computational load compared to centralized implementations. In a distributed network reducing computing time is a priority, allowing for more frequently updated actions and, therefore, more useful measurements. Performing multi-sensor control where multiple vehicles' actions are considered requires evaluating a large number of combinations of individual vehicle actions. A primary way to reduce computation time is to reduce the search space of the multi-sensor control commands. This can be achieved through numerous methods, including genetic algorithms (GA) [4].

Our proposed solutions are for multi-sensor control scenarios using a distributed sensor network with distributed information fused. This is achieved using the Random Finite Set (RFS) framework, a Bayesian framework for multi-target tracking, where the target states and observations are represented as random finite sets.

This paper evaluates the applicability and efficacy of multi-sensor control methods based on genetic algorithms, specifically for large and dynamically evolving vehicle networks. Several genetic algorithm formulations were developed and tested on a range of simulated environments. Both the multi-target tracking accuracy and the computation time of the proposed methods were evaluated and compared against baseline methods. This comparison explores the advantages and limitations of our proposed GA-based multi-sensor control methods in the context of large, dynamic vehicle networks.

2 RELATED WORKS

Numerous solutions have been developed for multi-target tracking-related problems using the RFS framework in the fields of sensor control in target tracking applications [8] and information fusion. Labeled RFS filters append the label of each target into its single-target state, allowing for the direct creation of target trajectories [11]. Several different information fusion approaches have been explored for applications using LMB filters for multi-target tracking, including Cooperative Cauchy-Schwarz divergence-based fusion, Consensus-based fusion, and Complementary fusion [6].

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

GECCO '24 Companion, July 14–18, 2024, Melbourne, VIC, Australia

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0495-6/24/07

<https://doi.org/10.1145/3638530.3654261>

Several optimization techniques can be used in multi-sensor control applications. Coordinate Descent has been used for multi-sensor control in RFS-based multi-target tracking applications in distributed implementations[3]. A distributed multi-sensor control algorithm that uses flooding-based communication has recently been proposed [2]. GAs have recently been used in single-sensor control applications for UAVs [5]. GAs have been utilized in the context of centralized multi-sensor control. This approach integrates into a Random Finite Set (RFS)-based filter designed for the concurrent tracking and searching of multiple targets, a significant stride in sensor network optimization [9].

3 MULTI-SENSOR CONTROL OVERVIEW

A multi-object state can be described by a labeled RFS composed of several single-object states that have been augmented with a label, $\mathbf{X} = \{(x, \ell) | x \in \mathbb{X}, \ell \in \mathbb{L}\}$ where $\mathbb{X}$ and $\mathbb{L}$ denote the spaces of single-object states and object labels, respectively.

The probability of detection of an object with state x is denoted by $p_{s,k}^d(x)$, which is dependent on the object's state, the state of the observing sensor, and the sensor's FoV. Sensors' FoVs are limited both by a restricted viewing angle (within $\theta_{\max}$ radians away from the sensor's principal axis) and a maximum detection range (within $\rho_{\max}$ m from the sensor). For a sensor s located at $(p_{x,s}, p_{y,s})$, the probability of detection of an object located at (p_x, p_y) is:

$$p_s^d(\rho, \theta) = \begin{cases} p_{\max}^d \frac{\tanh\left(\frac{[\rho_{\max} - \rho]}{\lambda}\right)}{\tanh\left(\frac{[\rho_{\max} - \rho_{\min}]}{\lambda}\right)} & \text{if } |\theta| \leq \theta_{\max}, \rho \leq \rho_{\max} \\ 0 & \text{else} \end{cases} \quad (1)$$

where

$$\rho = \sqrt{(p_x - p_{x,s})^2 + (p_y - p_{y,s})^2} \quad (2)$$

$$\theta = \text{atan2}(p_x - p_{x,s}, p_y - p_{y,s}) + \frac{\pi}{2} - \theta_s \quad (3)$$

At each discrete time k , the multi-object state is partially detected by a network of sensors. Each sensor s returns a set of measurements $\mathbf{Z}_s = \{z|x \in \mathbb{Z}\}$, where $\mathbb{Z}$ is the space of measurements, that are distributed according to a density given by the likelihood function $g_s(\cdot|x, x_s)$ where x is the detected object's state, and x_s denotes the sensor's state (e.g. its location $(p_{x,s}, p_{y,s})$ and orientation θ_s).

The sensors in a network that can communicate with a sensor s , known as the *neighbors* of s , is denoted $N(s)$. Each sensor can be controlled by executing a sensor control command representing a possible action $u \in \mathbb{U}$, where $\mathbb{U}$ is a finite set of sensor control commands. A multi-sensor control command $\mathbf{u}$ can be constructed by appending multiple single-sensor control commands.

3.1 Multi-Object Bayes Filter

The probability density of a multi-object state can be recursively estimated via a multi-object Bayes filter. The density of a multi-object state at time k is denoted by $\pi_k(\mathbf{X}_k | \mathbf{Z}_{1:k})$, where $\mathbf{Z}_{1:k}$ is the set of all measurements up to time k . In Bayesian filtering, in each time k the density undergoes both prediction and update. The predicted

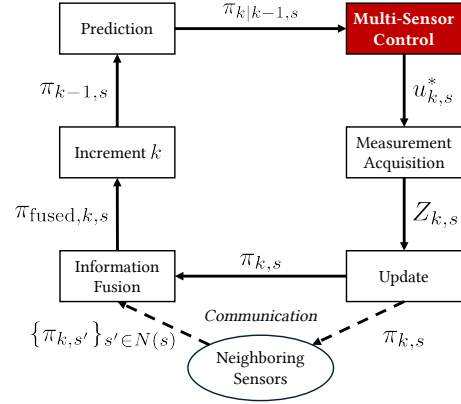


Figure 1: Overall operations running onboard each sensor node s , for tracking and control.

density is computed using the Chapman-Kolmogorov equation,

$$\pi_{k|k-1}(\mathbf{X}_k | \mathbf{Z}_{1:k-1}) = \int \pi_{k-1}(\mathbf{X}_{k-1} | \mathbf{Z}_{1:k-1}) f(\mathbf{X}_k | \mathbf{X}_{k-1}) \delta \mathbf{X}_{k-1} \quad (4)$$

After new measurements $\mathbf{Z}_k$ are acquired, the posterior density is computed from the prediction via the multi-object Bayes rule,

$$\pi_k(\mathbf{X}_k | \mathbf{Z}_{1:k}) = \frac{g(\mathbf{Z}_k | \mathbf{X}_k, \mathbf{u}_k) \pi_{k|k-1}(\mathbf{X}_k)}{\int g(\mathbf{Z}_k | \mathbf{X}, \mathbf{u}_k) \pi_{k|k-1}(\mathbf{X}) \delta \mathbf{X}} \quad (5)$$

A multi-object estimate of the existence and states of objects can be extracted from the posterior density.

While any labeled RFS-based multi-object filter could be used with the proposed multi-sensor control methods, the reliable and intuitive Labeled Multi-Bernoulli (LMB) filter was chosen. To mitigate the limitation of restricted sensor FoVs, we utilize Klupacs et al.'s distributed complementary fusion [7].

3.2 Overall Architecture

Figure 1 provides an overview of the entire distributed multi-sensor control problem from the perspective of a single sensor s . At each discrete time k , the cycle of prediction, control, measurement acquisition, update, and information fusion is carried out. We focus on the multi-sensor control step. Determining which control commands should be applied is a combinatorial optimization task. The significant computational cost of evaluating potential solutions requires the use of an efficient optimization scheme, such as GA.

4 GA MULTI-SENSOR CONTROL

4.1 Multi-Sensor Control Command

In the proposed multi-sensor control methods, each sensor aims to find jointly optimal actions for itself and its neighbors. The solutions that are considered locally at each sensor s are the possible multi-sensor control commands $\mathbf{u}_s = \{u_{s'} \in \mathbb{U}\}_{s' \in N(s) \cup \{s\}}$. Each multi-sensor control command generates a different fused pseudo-updated posterior and, therefore, a different value for the objective function (6). Once the optimal multi-sensor control command is found at each sensor s , $\mathbf{u}_s^* = \{u_{s'}^*\}_{s' \in N(s) \cup \{s\}}$, the single-sensor

control command for sensor s in $\mathbf{u}_s^*$, $\mathbf{u}_s^*$, is selected and then executed to move the sensor before measurement acquisition, enabling cooperation between sensors while reducing computational cost.

4.2 Objective Function

The optimal multi-sensor control command maximizes the reward function $v(\cdot)$, selected as the Kullback-Leibler Divergence between the prediction and the fused pseudo-update, with an additional penalty term ζ for dropped labels, used in previous work [2, Eq. 11]:

$$v\left(\pi_{k|k-1,s}(), \tilde{\pi}_{\text{fused},k,s}(\cdot; \mathbf{u}_s)\right) = D_{\text{KL}}\left(\tilde{\pi}_{\text{fused},k,s}(\cdot; \mathbf{u}_s) \parallel \pi_{k|k-1,s}()\right) + \zeta \quad (6)$$

4.3 Distributed Genetic Algorithm-Based Multi-Sensor Control

GAs establish a population of candidate solutions that undergo an iterative process of selection, crossover and mutation, to rapidly identify optimal solutions. For the multi-sensor control problem, each sensor s has a population of candidate solutions (or samples), denoted by $\mathbf{U}_s$, where each sample is a possible multi-sensor control command for itself and its neighbors, $\mathbf{u}_s = \{\mathbf{u}_{s'}\}_{s' \in N(s) \cup \{s\}}$. To encode a multi-sensor control command into a sample, first, the possible single-sensor commands $\mathbf{u} \in \mathcal{U}$ are coded into an integer, such that each integer from 1 to $|\mathcal{U}|$ encodes a unique control command. The multi-sensor control command $\mathbf{u}_s$ is encoded as a concatenation of the single-sensor control commands for s and its neighbors, $\{\mathbf{u}_{s'}\}_{s' \in N(s) \cup \{s\}}$, forming a string of integers.

The proposed methods vary in two key steps in the genetic algorithm process: reproduction and crossover. *Reproduction* selects the most optimal samples to continue to the next iteration. Multiple methods can be used for selection during reproduction; in this work, the following two schemes have been selected:

- **Roulette Wheel Selection:** Each sample from the original population has a probability of being selected as each new candidate solution based on its reward, calculated using $p(\mathbf{u}_s) = v(\mathbf{u}_s) / \sum_{\mathbf{u}'_s \in \mathcal{U}} v(\mathbf{u}'_s)$.
- **Tournament Selection:** Each new candidate solution is chosen through a ‘tournament’, where m samples are chosen with uniform probability from the original population and the optimal sample is selected as the new candidate solution.

Several different *crossover* techniques can be applied; in this work, the following two schemes have been selected:

- **Single-Point Crossover:** An index is uniformly selected from 1 to $|N(s)|+1$, the ‘children’ are formed by swapping all integers at that index and after between the two samples.
- **Uniform Crossover:** At each index from 1 to $|N(s)|+1$ one ‘child’ receives the integer at that index from one of the ‘parents’ and the other ‘child’ receives the integer at that index of the other ‘parent’, with uniform probability.

We propose four GA-based multi-sensor control methods using the combinations of Roulette wheel selection, Tournament selection, Single-point Crossover and Uniform crossover: GA-SR-SC, GA-ST-SC, GA-UR-SC, and GA-UT-SC.

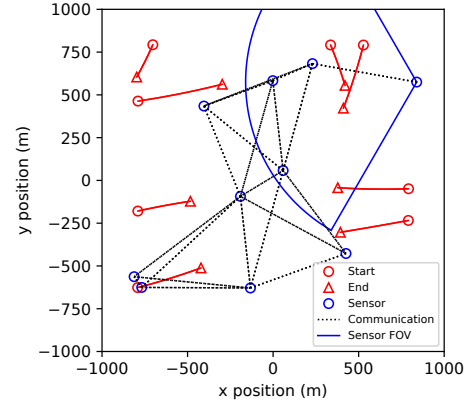


Figure 2: Second scenario with random initial sensor states.

5 EXPERIMENTAL RESULTS

5.1 Methods & Scenarios

The multi-target tracking accuracy of the four listed GA-based multi-sensor control methods is compared against two baselines: fixed camera positions (with no control) and a distributed flooding RFS-based multi-sensor control algorithm (DF-SC) [3].

For all four GA-based methods, a population size of 16 (in the first scenario) and 30 (in the second scenario) is chosen, and the maximum number of iterations is set to 5. 30% of the samples after crossover are selected as mutation candidates, with a mutation rate of 0.03 for each string index. The tournament size m is set to 3.

Two challenging simulated scenarios are used to evaluate each method’s multi-target tracking performance. Both scenarios take place within a $2000 \text{ m} \times 2000 \text{ m}$ area, tracking eight targets moving within a $1600 \text{ m} \times 1600 \text{ m}$ target area over a total of 50 time steps. Target births occur randomly along the edges of the target area. Each target has a probability of death $p_{\text{death}} = 0.005$ at each time step after 25. These targets follow a constant turn motion model, where the turn rate is uniformly distributed in $\pm 0.25^\circ/\text{sec}$. Note that each scenario has different targets.

The first scenario has 15 sensors with $\theta_{\text{max}} = 90^\circ$ (180° FoV) and maximum detection range of $\rho_{\text{max}} = 600 \text{ m}$. The communication range between sensors is 600 m . Each sensor can either remain stationary, rotate 22.5° clockwise, and rotate 22.5° anticlockwise. The second scenario has 10 sensors with $\theta_{\text{max}} = 60^\circ$ (120° FoV) with a maximum detection range of $\rho_{\text{max}} = 1000 \text{ m}$. The communication range between sensors is 800 m . In addition to the above three actions, each sensor can move 10 m up, 10 m left, 10 m down, and 10 m right, for seven total possible sensor control commands.

In both scenarios, the p_{max}^d and λ parameters in equation (1) are set at 0.995 and 65 m . Each measurement is subject to noise, with the white measurement noise variance along each axis increasing with the sensor-object distance, ρ , at a rate of $1 + (\rho/1000 \text{ m})^2$. False detections are uniformly distributed over the target area, with the number modeled by a Poisson distribution parameterized by $\lambda = 5$.

In both scenarios, 30 Monte Carlo experiments are performed with random initial sensor states in each experiment. Figure 2 shows a sample of random sensor initialization in the second scenario.

Table 1: Comparison of tracking performance results.

Experiment	OSPA error	OSPA ⁽²⁾ error	Time (s)
Fixed, 1 st	33.81 (17.57)	33.12 (14.95)	1.53e-07
DF-SC, 1 st	29.93 (16.49)	33.26 (15.82)	12.29
GA-SR-SC, 1 st	32.16 (21.99)	32.77 (17.61)	2.45
GA-ST-SC, 1 st	30.61 (19.00)	34.09 (16.19)	2.46
GA-UR-SC, 1 st	28.65 (20.63)	32.25 (18.49)	2.46
GA-UT-SC, 1 st	27.67 (15.79)	30.58 (15.87)	2.80
Fixed, 2 nd	31.24 (16.44)	38.02 (14.54)	1.47e-07
DF-SC, 2 nd	28.82 (14.06)	36.65 (12.26)	3.13
GA-SR-SC, 2 nd	28.18 (15.38)	36.17 (13.18)	8.96
GA-ST-SC, 2 nd	29.69 (17.42)	37.64 (13.63)	8.53
GA-UR-SC, 2 nd	26.40 (16.93)	33.26 (14.51)	10.06
GA-UT-SC, 2 nd	22.35 (13.20)	30.46 (14.76)	8.61

5.2 Results

Table 1 details the multi-target tracking performance of the methods on both scenarios, evaluated using the OSPA [10] and OSPA⁽²⁾ [1] error metrics. These metrics consider both the estimated states of the targets and the number of estimated targets. Additionally, OSPA⁽²⁾ accounts for errors between target tracks. The mean and standard deviation (in brackets) of these errors (across all sensors and all simulation runs), as well as the mean computation time per sensor per run, are listed for all methods and scenarios.

5.3 Discussion

In both scenarios the GA-based methods had improved tracking accuracy compared to Fixed cameras, except for the OSPA⁽²⁾ error of GA-ST-SC in the first scenario. The two single-point crossover methods performed worse than DF-SC in the first scenario and similarly in the second, whereas the two uniform crossover methods performed significantly better than DF-SC, particularly GA-UT-SC. Additionally, the GA-based methods had $\sim 5\times$ faster computation time than the DF-SC method in the first scenario, highlighting the efficiency that genetic algorithms can provide in scenarios with many sensors. However in the second scenario the GA-based methods had $\sim 2.7\text{--}3.2\times$ slower computation time than DF-SC, due to the increase in population size to achieve high tracking accuracy.

Paired t-tests were performed over the OSPA and OSPA⁽²⁾ errors of the DF-SC method and the GA-based methods. In both scenarios GA-UT-SC outperforms DF-SC statistically, with p -values for the OSPA error of 0.0497 and 6.65e-08 and p -values for the OSPA⁽²⁾ error of 0.0169 and 6.24e-08 in scenarios 1 and 2 respectively. In the second scenario the p -values between GA-UR-SC and DF-SC are 0.0480 for the OSPA error and 0.0015 for the OSPA⁽²⁾ error.

In both scenarios the combination of uniform crossover and tournament selection is the best performing. It is hypothesized that uniform crossover tends to create more varied ‘children’ candidates than single-point crossover, allowing for a more varied search, in addition to tournament selection being more likely to select the better-performing candidates compared to roulette wheel selection.

6 CONCLUSION

This paper addresses the problem of multi-sensor control in distributed multi-target tracking applications, focusing on comparing

GA-based methods to existing methods. The tracking performance of these methods using the OSPA and OSPA⁽²⁾ errors was evaluated on two scenarios with varying sensor numbers, detection profiles, and target trajectories.

The experiments found that all four GA-based methods performed better than the fixed cameras in both scenarios and were similar or better to DF-SC, using both error metrics, with the GA-UT-SC method performing best. Additionally, the computation time of the GA-based methods was significantly reduced compared to DF-SC in the first scenario, but increased in the second scenario. This paper shows that GAs are a strong optimization method in the multi-sensor control problem for distributed multi-target tracking, however further experiments with a wider range of scenarios and more sophisticated methods would strengthen these findings.

ACKNOWLEDGMENTS

The Australian Research Council supported this work through Grant DE210101181.

REFERENCES

- [1] Michael Beard, Ba Tuong Vo, and Ba-Ngu Vo. 2017. OSPA(2): Using the OSPA metric to evaluate multi-target tracking performance. In *2017 International Conference on Control, Automation and Information Sciences (ICCAIS)*. 86–91. <https://doi.org/10.1109/ICCAIS.2017.8217598>
- [2] Aidan Blair, Amirali Khodadadian Gostar, Alireza Bab-Hadiashar, Xiaodong Li, and Reza Hoseinnezhad. 2024. Enhanced Multi-Target Tracking in Dynamic Environments: Distributed Control Methods Within the Random Finite Set Framework. *arXiv:2401.14085 [eess.SY]*
- [3] Aidan Blair, Amirali Khodadadian Gostar, Ruwan Tennakoon, Alireza Bab-Hadiashar, Xiaodong Li, Jennifer Palmer, and Reza Hoseinnezhad. 2022. Distributed Multi-Sensor Control for Multi-Target Tracking. In *International Conference on Control, Automation and Information Sciences (ICCAIS 2022)*. 72. <https://doi.org/10.1117/12.2637606>
- [4] David E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, New York.
- [5] Lianghao Hua, Jianfeng Zhang, Dejie Li, Xiaobo Xi, and Mohd Shah. 2022. Sensor Fault Diagnosis and Fault Tolerant Control of Quadrotor UAV Based on Genetic Algorithm. *Journal of Sensors* 2022 (03 2022), 1–8. <https://doi.org/10.1155/2022/8626722>
- [6] James Klupacs, Amirali Khodadadian Gostar, Tharindu Rathnayake, Iqbal Gondal, Alireza Bab-Hadiashar, and Reza Hoseinnezhad. 2022. Multi-Agent Information Fusion for Connected Driving: A Review. *IEEE Access* (2022), 1–1. <https://doi.org/10.1109/ACCESS.2022.3197665>
- [7] James Klupacs, Amirali Khodadadian Gostar, Alireza Bab-Hadiashar, Jennifer Palmer, and Reza Hoseinnezhad. 2022. Distributed Complementary Fusion for Connected Vehicles. In *2022 11th International Conference on Control, Automation and Information Sciences (ICCAIS)*. 316–321. <https://doi.org/10.1109/ICCAIS56082.2022.9990319>
- [8] Sabita Panicker, Amirali Khodadadian Gostar, Alireza Bab-Hadiashar, and Reza Hoseinnezhad. 2020. Tracking of targets of interest using labeled multi-Bernoulli filter with multi-sensor control. *Signal Processing* 171 (2020), 107451. <https://doi.org/10.1016/j.sigpro.2019.107451>
- [9] Savvas Papaioannou, Panayiotis Kolios, Theodoris Theodorides, Christos G. Panayiotou, and Marios M. Polycarpou. 2020. Jointly-Optimized Searching and Tracking with Random Finite Sets. *IEEE Transactions on Mobile Computing* 19, 10 (2020), 2374–2391. <https://doi.org/10.1109/TMC.2019.2922133>
- [10] Dominic Schuhmacher, Ba-Tuong Vo, and Ba-Ngu Vo. 2008. A Consistent Metric for Performance Evaluation of Multi-Object Filters. *IEEE Transactions on Signal Processing* 56, 8 (2008), 3447–3457. <https://doi.org/10.1109/TSP.2008.920469>
- [11] Ba Tuong Vo and Ba Ngu Vo. 2013. Labeled random finite sets and multi-object conjugate priors. *IEEE Transactions on Signal Processing* 61, 13 (2013), 3460–3475. <https://doi.org/10.1109/TSP.2013.2259822> *arXiv:arXiv:1312.2372v1*
- [12] Tan-Lin Xiao, Qiang Yang, Xu-Dong Gao, Yuan-Yuan Ma, Zhen-Yu Lu, Sang-Woon Jeon, and Jun Zhang. 2023. Variation Encoded Large-Scale Swarm Optimizers for Path Planning of Unmanned Aerial Vehicle. In *Proceedings of the Genetic and Evolutionary Computation Conference (Lisbon, Portugal) (GECCO '23)*. Association for Computing Machinery, New York, NY, USA, 102–110. <https://doi.org/10.1145/3583131.3590357>

Received 1 February 2024; accepted 11 April 2024